

## MLDL EXPERIMENT 7

### AIM -

Build an Artificial Neural Network (ANN) using Keras/TensorFlow-

### DATASET SOURCE -

<https://www.kaggle.com/datasets/shrutimechlearn/churn-modelling>

### DATASET DESCRIPTION -

This data set contains details of a bank's customers and the target variable is a binary variable reflecting the fact whether the customer left the bank (closed his account) or he continues to be a customer.

#### 1. Overview

The dataset used in this experiment is a **Bank Customer Churn Dataset**, which contains information about customers of a bank. The purpose of this dataset is to predict whether a customer will **leave the bank (churn)** or **continue using the services**.

This is a **supervised machine learning dataset** used for **binary classification**.

#### 2. Objective of Dataset

The dataset helps in:

- Understanding customer behavior
- Identifying patterns leading to churn
- Building a predictive model (ANN) to classify customers

#### 3. Dataset Size

- Total Records: **10,000 customers**
- Total Features: **14 columns**

## 4. Target Variable

### Exited

- 0 → Customer stays
- 1 → Customer leaves (churn)

This is the **dependent/output variable**.

## 5. Feature Description (Input Variables)

### 1. RowNumber

- Unique index for each row
- Not useful for prediction → removed

### 2. CustomerId

- Unique ID of customer
- Not useful → removed

### 3. Surname

- Customer name
- Not useful → removed

### 4. CreditScore

- Numerical value representing creditworthiness
- Higher score → less likely to churn

### 5. Geography

- Country of customer (France, Germany, Spain)
- Categorical feature
- Converted using encoding

### 6. Gender

- Male / Female
- Categorical → encoded

## **7. Age**

- Age of customer
- Important factor in churn prediction

## **8. Tenure**

- Number of years customer has stayed with bank

## **9. Balance**

- Account balance
- Higher balance → usually lower churn

## **10. NumOfProducts**

- Number of bank products used
- Example: savings account, credit card

## **11. HasCrCard**

- Whether customer has credit card
- 1 → Yes, 0 → No

## **12. IsActiveMember**

- Whether customer is active
- 1 → Active, 0 → Inactive

## **13. EstimatedSalary**

- Estimated yearly income

## **14. Exited (Target Variable)**

- 1 → Customer left
- 0 → Customer stayed

## THEORY -

### 1. Introduction to Artificial Neural Network (ANN)

Artificial Neural Network (ANN) is a computational model inspired by the functioning of the **human brain's biological neurons**. It is one of the most important techniques in machine learning and deep learning used for solving complex problems such as classification, regression, image recognition, and natural language processing.

An ANN consists of a large number of interconnected processing elements called **neurons**, which work together to process information and learn patterns from data.

In this experiment, ANN is used for **binary classification**, where the goal is to predict whether a bank customer will **leave the bank (churn)** or **stay** based on their personal and financial attributes.

### 2. Biological Inspiration

The human brain consists of billions of neurons connected through synapses. Each neuron:

- Receives input signals
- Processes them
- Produces an output signal

Similarly, in ANN:

- Inputs are features of data
- Neurons process weighted inputs
- Output is prediction

### 3. Architecture of ANN

An Artificial Neural Network is composed of three main types of layers:

#### 3.1 Input Layer

- First layer of the network
- Receives input data (features)
- Each neuron represents one feature
- Inputs include Age, Balance, Salary, etc.

### 3.2 Hidden Layers

- Intermediate layers between input and output
- Perform computations and feature extraction
- Can be one or more layers

More hidden layers = deeper network

### 3.3 Output Layer

- Produces final result
- In binary classification:
  - Single neuron
  - Uses sigmoid activation

## 4. Mathematical Model of ANN

Each neuron performs the following computation:

$$Z = \sum_{i=1}^n W_i X_i + b$$

Where:

- $X_i$  = input features
- $W_i$  = weights
- $b$  = bias

Then activation function is applied:

$$A = f(Z)$$

## 5. Activation Functions

Activation functions introduce **non-linearity** into the model.

### 5.1 ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Used in hidden layers
- Efficient and fast
- Helps avoid vanishing gradient problem

### 5.2 Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

- Used in output layer
- Converts output into probability (0 to 1)

## 6. Working of ANN

ANN works in three main steps:

### 6.1 Forward Propagation

- Input data is passed through layers
- Each neuron calculates output
- Final output is generated

### 6.2 Loss Calculation

- Difference between predicted and actual output is calculated
- In this experiment:

Loss = Binary Cross Entropy

### 6.3 Backpropagation

- Error is propagated backward
- Weights are updated using gradient descent

## 7. Training Process

The training process involves:

### Step 1: Initialize Weights

- All weights are assigned **random small values**
- Why?
  - If all weights were same → network learns nothing

### Step 2: Forward Propagation

Data flows like this:

Input → Hidden Layer → Output

Example:

- Input: Age = 40, Balance = 60000
- Each input is multiplied by weights
- Passed through activation functions
- Final output = probability (e.g., 0.78 → customer may leave)

### Step 3: Loss Calculation

We compare:

- Actual output → 1 (customer left)
- Predicted → 0.78

Loss measures error.

In your experiment:

- **Binary Cross Entropy Loss**

### Step 4: Backpropagation

This is the **main learning step**

- Error is sent backward
- Network checks:
  - “Which weights caused the error?”
- Adjusts weights accordingly

### Step 5: Weight Update

Using optimizer (Adam):

$$\text{New Weight} = \text{Old Weight} - \text{Learning Rate} \times \text{Gradient}$$

### Step 6: Repeat (Epochs)

- Whole dataset is passed multiple times
- Each cycle = 1 **epoch**

Example:

- Epoch 1 → Accuracy = 70%
- Epoch 20 → Accuracy = 85%

## 8. Hyperparameters in ANN

Hyperparameters are parameters set before training.

**Main Hyperparameters in your experiment:**

### 8.1. Epochs

- Number of training cycles

Effect:

- Too low → underfitting
- Too high → overfitting

### 8.2. Batch Size

- Number of samples processed at once

Example:

- Dataset = 1000 rows
- Batch size = 32 → 32 rows processed at a time

Effect:



- Small batch → better learning but slow
- Large batch → fast but less accurate

### 8.3. Learning Rate

- Controls how much weights change

Effect:

- Too high → unstable model
- Too low → very slow learning

### 8.4. Number of Neurons

- More neurons → more learning power
- Too many → overfitting

### 8.5. Hidden Layers

- More layers = deep learning
- Too deep → complex + slow

## 9. Performance Evaluation

The model is evaluated using:

### 9.1. Accuracy

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

### 9.2. Confusion Matrix

Shows:

- True Positives
- True Negatives
- False Positives

- False Negatives

### **9.3. Loss Curve**

Shows error reduction during training

### **9.4. Accuracy Curve**

Shows improvement in prediction accuracy

## **10. Graphical Analysis**

Graphs used in this experiment:

- Accuracy vs Epoch
- Loss vs Epoch
- Confusion Matrix
- Feature Correlation Heatmap
- Class Distribution
- Prediction Probability

## **11. Advantages of ANN**

- Can model complex non-linear relationships
- High prediction accuracy
- Learns automatically from data
- Flexible architecture

## **12. Limitations of ANN**

- Requires large dataset
- Computationally expensive
- Difficult to interpret (black box)
- Prone to overfitting

## **13. Applications of ANN**

- Customer churn prediction
- Image recognition
- Speech recognition
- Medical diagnosis
- Fraud detection

## CODE -

```
#
=====
=====
# IMPORT LIBRARIES
#
=====
=====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import
LabelEncoder, StandardScaler
from sklearn.model_selection import
train_test_split
from sklearn.metrics import confusion_matrix,
accuracy_score, classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

#
=====
=====
# LOAD DATASET
#
=====
=====
df =
pd.read_csv('/content/Churn_Modelling.csv')

#
=====
=====
# PREPROCESSING
```

```
#
=====
=====
X = df.iloc[:, 3:13]
y = df.iloc[:, 13]

le = LabelEncoder()
X['Gender'] = le.fit_transform(X['Gender'])

X = pd.get_dummies(X, drop_first=True)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

#
=====
=====
# SPLIT
#
=====
=====
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

#
=====
=====
# HYPERPARAMETER TUNING (MANUAL)
#
=====
=====
configs = [
    {'neurons':16, 'epochs':50},
    {'neurons':32, 'epochs':50},
    {'neurons':16, 'epochs':100}
]
```

```

results = []

for config in configs:
    model = Sequential()
    model.add(Dense(config['neurons'],
activation='relu', input_dim=X_train.shape[1]))
    model.add(Dense(config['neurons']//2,
activation='relu'))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['accuracy'])

    history = model.fit(X_train, y_train,
epochs=config['epochs'],
batch_size=32, verbose=0,
validation_data=(X_test, y_test))

    loss, acc = model.evaluate(X_test, y_test,
verbose=0)

    results.append((config, acc))
    print(f"Config: {config} → Accuracy: {acc}")

#
=====
====
# BEST MODEL
#
=====
====
best_config = max(results, key=lambda x:
x[1])[0]

print(f"🟢 BEST CONFIG:", best_config)

model = Sequential()
model.add(Dense(best_config['neurons'],
activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(best_config['neurons']//2,
activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train,
epochs=best_config['epochs'],
batch_size=32,

```

```

validation_data=(X_test, y_test))

#
=====
====
# PREDICTION
#
=====
====
y_pred = (model.predict(X_test) > 0.5)

acc = accuracy_score(y_test, y_pred)
print(f"🚀 FINAL ACCURACY:", acc)

print(f"📊 CLASSIFICATION REPORT:\n")
print(classification_report(y_test, y_pred))

#
=====
====
# CONFUSION MATRIX
#
=====
====
cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()

#
=====
====
# GRAPH 1: ACCURACY CURVE
#
=====
====
plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy vs Epoch")
plt.legend(['Train', 'Validation'])
plt.show()

#
=====
====
# GRAPH 2: LOSS CURVE

```

```

#
=====
=====
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss vs Epoch")
plt.legend(['Train', 'Validation'])
plt.show()

#
=====
=====
# GRAPH 3: CLASS DISTRIBUTION
#
=====
=====
plt.figure()
sns.countplot(x=y)
plt.title("Class Distribution")
plt.show()

#
=====
=====
# GRAPH 4: CORRELATION HEATMAP
#
=====
=====
plt.figure(figsize=(10,8))
# Select only numeric columns for correlation
heatmap
# Drop 'RowNumber', 'CustomerId', 'Surname'
as they are not numeric
df_numeric = df.drop(columns=['RowNumber',
'CustomerId', 'Surname'])
# One-hot encode 'Gender' and 'Geography' for
the correlation heatmap
# df_numeric['Gender'] needs to be encoded
similarly to how X['Gender'] was
# And 'Geography' needs one-hot encoding
df_numeric['Gender'] =
le.fit_transform(df_numeric['Gender'])
df_numeric = pd.get_dummies(df_numeric,
drop_first=True, columns=['Geography'])
sns.heatmap(df_numeric.corr(numeric_only=True),
annot=True)
plt.title("Feature Correlation")
plt.show()

```

```

#
=====
=====
# GRAPH 5: FEATURE IMPORTANCE
(APPROX)
#
=====
=====
weights = model.layers[0].get_weights()[0]
importance = np.mean(np.abs(weights), axis=1)

plt.figure()
plt.bar(range(len(importance)), importance)
plt.title("Feature Importance Approx")
plt.show()

#
=====
=====
# GRAPH 6: PREDICTION PROBABILITY
#
=====
=====
probs = model.predict(X_test)

plt.figure()
plt.hist(probs)
plt.title("Prediction Probability Distribution")
plt.show()

#
=====
=====
# 🔥 MANUAL INPUT PREDICTION
#
=====
=====
print("\n ♦ ENTER CUSTOMER DETAILS:")

sample = np.array([[600, 1, 40, 3, 60000, 2, 1, 1,
50000, 0, 1]])

sample_scaled = scaler.transform(sample)

pred = model.predict(sample_scaled)

print("\nProbability of Leaving:", pred[0][0])

```

```
if pred > 0.5:
    print("❌ Customer will LEAVE")
else:
    print("✅ Customer will STAY")
```

OUTPUT -

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Config: {'neurons': 16, 'epochs': 50} → Accuracy: 0.8504999090940094
Config: {'neurons': 32, 'epochs': 50} → Accuracy: 0.8654999732971191
Config: {'neurons': 16, 'epochs': 100} → Accuracy: 0.8579999804496765

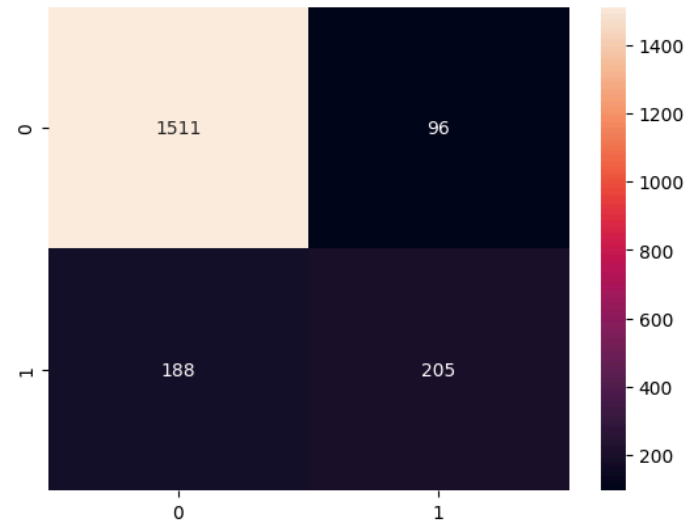
✅ BEST CONFIG: {'neurons': 32, 'epochs': 50}
Epoch 1/50
250/250 ————— 2s 3ms/step - accuracy: 0.7984 - loss: 0.4608 - val_accuracy: 0.8175 - val_lo
Epoch 2/50
250/250 ————— 1s 2ms/step - accuracy: 0.8264 - loss: 0.4093 - val_accuracy: 0.8315 - val_lo
Epoch 3/50
250/250 ————— 1s 2ms/step - accuracy: 0.8376 - loss: 0.3851 - val_accuracy: 0.8440 - val_lo
Epoch 4/50
250/250 ————— 1s 2ms/step - accuracy: 0.8486 - loss: 0.3666 - val_accuracy: 0.8525 - val_lo
Epoch 5/50
250/250 ————— 1s 2ms/step - accuracy: 0.8569 - loss: 0.3546 - val_accuracy: 0.8535 - val_lo
Epoch 6/50
250/250 ————— 1s 2ms/step - accuracy: 0.8590 - loss: 0.3463 - val_accuracy: 0.8545 - val_lo
Epoch 7/50
250/250 ————— 1s 2ms/step - accuracy: 0.8596 - loss: 0.3428 - val_accuracy: 0.8585 - val_lo
Epoch 8/50
250/250 ————— 1s 2ms/step - accuracy: 0.8622 - loss: 0.3386 - val_accuracy: 0.8590 - val_lo
Epoch 9/50
250/250 ————— 1s 3ms/step - accuracy: 0.8620 - loss: 0.3357 - val_accuracy: 0.8610 - val_lo
Epoch 10/50
250/250 ————— 1s 4ms/step - accuracy: 0.8626 - loss: 0.3339 - val_accuracy: 0.8595 - val_lo
Epoch 11/50
250/250 ————— 1s 4ms/step - accuracy: 0.8644 - loss: 0.3320 - val_accuracy: 0.8610 - val_lo
Epoch 12/50
250/250 ————— 1s 2ms/step - accuracy: 0.8658 - loss: 0.3296 - val_accuracy: 0.8610 - val_lo
Epoch 13/50
250/250 ————— 1s 2ms/step - accuracy: 0.8684 - loss: 0.3170 - val_accuracy: 0.8590 - val_lo
Epoch 14/50
250/250 ————— 1s 2ms/step - accuracy: 0.8686 - loss: 0.3159 - val_accuracy: 0.8585 - val_lo
Epoch 15/50
250/250 ————— 1s 2ms/step - accuracy: 0.8692 - loss: 0.3149 - val_accuracy: 0.8575 - val_lo
Epoch 16/50
250/250 ————— 1s 2ms/step - accuracy: 0.8686 - loss: 0.3146 - val_accuracy: 0.8600 - val_lo
Epoch 17/50
250/250 ————— 1s 3ms/step - accuracy: 0.8725 - loss: 0.3137 - val_accuracy: 0.8615 - val_lo
Epoch 18/50
250/250 ————— 1s 3ms/step - accuracy: 0.8689 - loss: 0.3140 - val_accuracy: 0.8550 - val_lo
Epoch 19/50
250/250 ————— 1s 3ms/step - accuracy: 0.8699 - loss: 0.3125 - val_accuracy: 0.8565 - val_lo
Epoch 20/50
250/250 ————— 1s 2ms/step - accuracy: 0.8700 - loss: 0.3124 - val_accuracy: 0.8570 - val_lo
Epoch 21/50
250/250 ————— 1s 2ms/step - accuracy: 0.8715 - loss: 0.3116 - val_accuracy: 0.8595 - val_lo
Epoch 22/50
250/250 ————— 1s 2ms/step - accuracy: 0.8716 - loss: 0.3120 - val_accuracy: 0.8605 - val_lo
Epoch 23/50
250/250 ————— 1s 2ms/step - accuracy: 0.8691 - loss: 0.3113 - val_accuracy: 0.8550 - val_lo
Epoch 24/50
250/250 ————— 1s 2ms/step - accuracy: 0.8706 - loss: 0.3110 - val_accuracy: 0.8585 - val_lo
Epoch 25/50
250/250 ————— 1s 2ms/step - accuracy: 0.8708 - loss: 0.3090 - val_accuracy: 0.8570 - val_lo
Epoch 26/50
250/250 ————— 1s 2ms/step - accuracy: 0.8714 - loss: 0.3089 - val_accuracy: 0.8555 - val_lo
Epoch 27/50
250/250 ————— 1s 2ms/step - accuracy: 0.8715 - loss: 0.3086 - val_accuracy: 0.8555 - val_lo
Epoch 28/50
250/250 ————— 1s 2ms/step - accuracy: 0.8719 - loss: 0.3081 - val_accuracy: 0.8615 - val_lo
Epoch 29/50
250/250 ————— 1s 2ms/step - accuracy: 0.8716 - loss: 0.3077 - val_accuracy: 0.8575 - val_lo
```

🚩 FINAL ACCURACY: 0.858

📊 CLASSIFICATION REPORT:

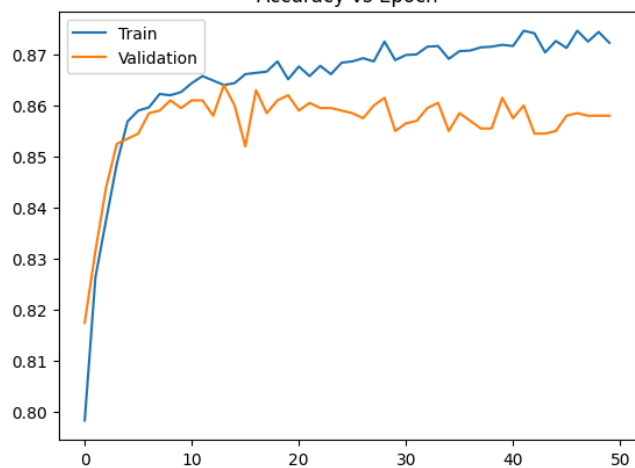
|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.89      | 0.94   | 0.91     | 1607    |
| 1            | 0.68      | 0.52   | 0.59     | 393     |
| accuracy     |           |        | 0.86     | 2000    |
| macro avg    | 0.79      | 0.73   | 0.75     | 2000    |
| weighted avg | 0.85      | 0.86   | 0.85     | 2000    |

Confusion Matrix

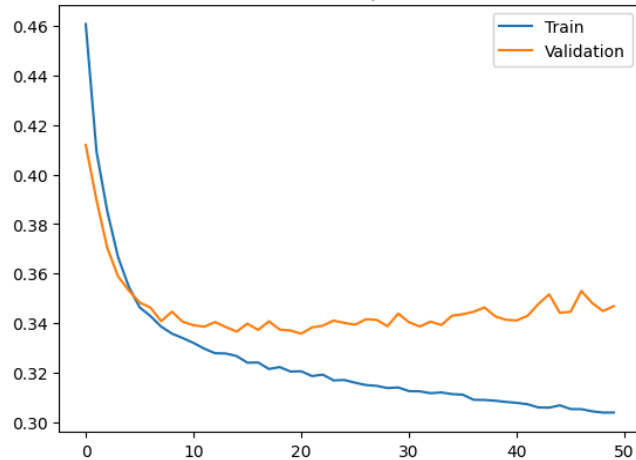


Accuracy vs Epoch

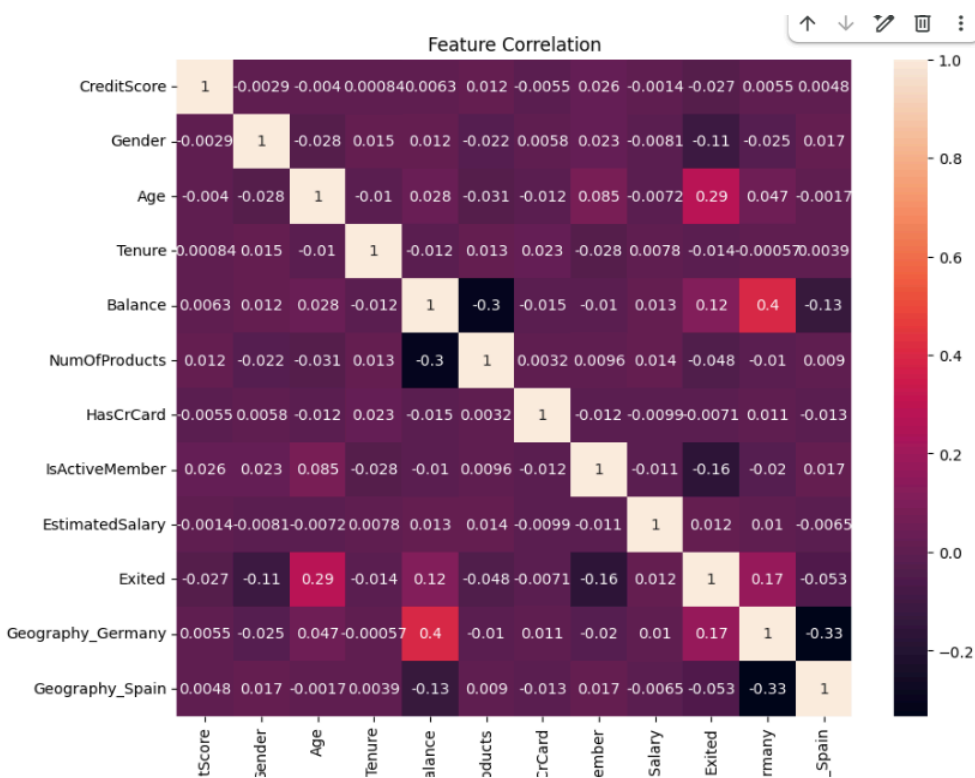
Accuracy vs Epoch

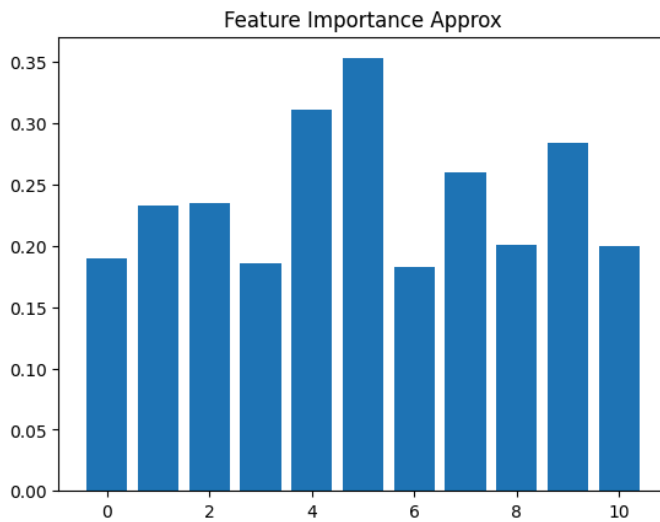


Loss vs Epoch



Feature Correlation





Probability of Leaving: 0.021744372

✅ Customer will STAY

```
/usr/local/lib/python3.12/dist-packages/sklearn/util  
warnings.warn(
```

## 18. Conclusion

Artificial Neural Network is a powerful machine learning technique capable of learning complex patterns from data. In this experiment, ANN was successfully used to predict customer churn. The model was trained using backpropagation and evaluated using accuracy and various visualization techniques, demonstrating its effectiveness in classification tasks.